

Entregable Final

Presentado por el estudiante:

Guillermo Andrés Saucedo Dávalos

Dirigido por el docente:

Ing. Jared Lopez Leños

Materia:

Tecnología de base de datos

Santa Cruz, Bolivia

2026

Github:

https://github.com/GuillermoSaucedo890/EntregableFinal_TecBasedeDatos_GUI
LLERMOSAUCEDO

Punto 1: Migración de Tablas

En esta etapa se realiza la exportación de los datos almacenados en MariaDB hacia archivos en formato CSV. Para ello, se utiliza la sentencia **SELECT ... INTO OUTFILE**, que permite ejecutar una consulta y guardar directamente el resultado en un archivo dentro del sistema de archivos del servidor de MariaDB.

La opción **FIELDS TERMINATED BY ' , '** establece que los campos estarán separados por comas, siguiendo el formato CSV. **OPTIONALLY ENCLOSED BY ' "'** indica que los valores pueden estar encerrados entre comillas dobles, lo que permite manejar correctamente datos que puedan contener caracteres especiales. Finalmente, **LINES TERMINATED BY '\n'** establece que cada registro se almacenará en una línea diferente.

Exportacion de datos de tabla:

```
SELECT * FROM departments  
  
INTO OUTFILE '/tmp/departments.csv'  
  
FIELDS TERMINATED BY ','  
  
OPTIONALLY ENCLOSED BY '\"'  
  
LINES TERMINATED BY '\n';
```

```
SELECT * FROM employees  
  
INTO OUTFILE '/tmp/employees.csv'  
  
FIELDS TERMINATED BY ','  
  
OPTIONALLY ENCLOSED BY '\"'  
  
LINES TERMINATED BY '\n';
```

```
SELECT * FROM dept_emp  
  
INTO OUTFILE '/tmp/dept_emp.csv'  
  
FIELDS TERMINATED BY ','  
  
OPTIONALLY ENCLOSED BY '\"'  
  
LINES TERMINATED BY '\n';
```

```
SELECT * FROM dept_manager
INTO OUTFILE '/tmp/dept_manager.csv'
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n';
```

```
SELECT * FROM salaries
INTO OUTFILE '/tmp/salaries.csv'
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n';
```

```
SELECT * FROM titles
INTO OUTFILE '/tmp/titles.csv'
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n';
```

A continuación se ejecuta el siguiente comando para verificar que los archivos .csv se crearon correctamente dentro del contenedor mariadb

```
└─[X]─[guillermo@saucedo]─[~]
└─ $ docker exec mariadb sh -c 'ls -lh /tmp/*.csv'
-rw-r--r-- 1 mysql mysql 189 Sep 27 08:05 /tmp/departments.csv
-rw-r--r-- 1 mysql mysql 13M Sep 27 08:22 /tmp/dept_emp.csv
-rw-r--r-- 1 mysql mysql 960 Sep 27 08:22 /tmp/dept_manager.csv
-rw-r--r-- 1 mysql mysql 17M Sep 27 08:20 /tmp/employees.csv
-rw-r--r-- 1 mysql mysql 106M Sep 27 08:23 /tmp/salaries.csv
-rw-r--r-- 1 mysql mysql 20M Sep 27 08:23 /tmp/titles.csv
```

Copiar .csv a postgres

Se pasan los archivos .csv al contenedor de postgres

```
docker cp mariadb:/tmp/departments.csv .
```

```
docker cp mariadb:/tmp/employees.csv .
```

```
docker cp mariadb:/tmp/dept_emp.csv .
```

```
docker cp mariadb:/tmp/dept_manager.csv .
```

```
docker cp mariadb:/tmp/salaries.csv .
```

```
docker cp mariadb:/tmp/titles.csv .
```

```
docker cp departments.csv postgresql:/tmp/departments.csv
```

```
docker cp employees.csv postgresql:/tmp/employees.csv
```

```
docker cp dept_emp.csv postgresql:/tmp/dept_emp.csv
```

```
docker cp dept_manager.csv postgresql:/tmp/dept_manager.csv
```

```
docker cp salaries.csv postgresql:/tmp/salaries.csv
```

```
docker cp titles.csv postgresql:/tmp/titles.csv
```

Migracion de datos

Se realiza la carga de los datos previamente exportados desde MariaDB hacia las tablas creadas en PostgreSQL. Para ello se utiliza la instrucción **COPY**, una herramienta propia de PostgreSQL que permite importar grandes volúmenes de información desde archivos externos de forma eficiente.

La instrucción **COPY** indica a PostgreSQL que debe insertar datos dentro de una tabla específica. La cláusula **FROM** define la ubicación del archivo CSV que contiene los registros a importar. Dentro de la opción **WITH** se especifican los parámetros de lectura del archivo: **FORMAT csv** establece que el archivo utiliza el formato **CSV**, **DELIMITER ','** indica que las columnas están separadas por comas y **QUOTE ''''** define que los valores pueden estar delimitados mediante comillas dobles.

```
pdb_employees=# COPY departments
```

```
FROM '/tmp/departments.csv'
```

```
WITH (  
    FORMAT csv,  
    DELIMITER ',',  
    QUOTE ''''  
);  
COPY 9  
pdb_employees=# COPY employees  
FROM '/tmp/employees.csv'  
WITH (  
    FORMAT csv,  
    DELIMITER ',',  
    QUOTE ''''  
);  
COPY 300024  
pdb_employees=# COPY dept_emp  
FROM '/tmp/dept_emp.csv'  
WITH (  
    FORMAT csv,  
    DELIMITER ',',  
    QUOTE ''''  
);  
COPY 331603  
pdb_employees=# COPY dept_manager  
FROM '/tmp/dept_manager.csv'  
WITH (  
    FORMAT csv,  
    DELIMITER ',',  
    QUOTE ''''  
);  
COPY 24
```

```

pdb_employees=# COPY salaries
FROM '/tmp/salaries.csv'
WITH (
    FORMAT csv,
    DELIMITER ',',
    QUOTE '"'
);
COPY 2844047
pdb_employees=# COPY titles
FROM '/tmp/titles.csv'
WITH (
    FORMAT csv,
    DELIMITER ',',
    QUOTE '"'
);
COPY 443308
pdb_employees=#

```

Conteo de filas

En esta etapa se realiza una comparación del número de registros existentes en las tablas de MariaDB y PostgreSQL, con el objetivo de verificar que la migración de datos se haya completado correctamente y que no existan pérdidas de información durante el proceso de transferencia.

Para realizar esta validación se utiliza una consulta con **COUNT(*)**, combinada con **UNION ALL** para obtener en un solo resultado el conteo de filas de todas las tablas migradas.

MariaDB:

```
MariaDB [employees]> SELECT 'departments' AS tabla, COUNT(*) AS filas FROM departments
```

```

-> UNION ALL
-> SELECT 'employees', COUNT(*) FROM employees
-> UNION ALL
-> SELECT 'dept_emp', COUNT(*) FROM dept_emp
-> UNION ALL
-> SELECT 'dept_manager', COUNT(*) FROM dept_manager
-> UNION ALL
-> SELECT 'salaries', COUNT(*) FROM salaries
-> UNION ALL
-> SELECT 'titles', COUNT(*) FROM titles;

```

```

+-----+-----+
| tabla   | filas  |
+-----+-----+
| departments | 9 |
| employees   | 300024 |
| dept_emp    | 331603 |
| dept_manager | 24 |
| salaries    | 2844047 |
| titles      | 443308 |
+-----+-----+
6 rows in set (2,843 sec)

```

Postgres:

```

pdb_employees=# SELECT 'departments' AS tabla, COUNT(*) AS filas FROM departments

```

UNION ALL

SELECT 'employees', COUNT(*) FROM employees

UNION ALL

SELECT 'dept_emp', COUNT(*) FROM dept_emp

UNION ALL

SELECT 'dept_manager', COUNT(*) FROM dept_manager

UNION ALL

SELECT 'salaries', COUNT(*) FROM salaries

UNION ALL

SELECT 'titles', COUNT(*) FROM titles;

tabla | filas

-----+-----

departments | 9

employees | 300024

dept_emp | 331603

dept_manager | 24

salaries | 2844047

titles | 443308

(6 filas)

Punto 2: Migración de Vistas

En esta etapa se realiza la migración de las vistas existentes en MariaDB hacia PostgreSQL.

Primero se identifican las vistas disponibles en MariaDB mediante el siguiente comando:

Vistas MariaDB

```
MariaDB [employees]> SHOW FULL TABLES WHERE TABLE_TYPE = 'VIEW';
```

```
+-----+-----+
| Tables_in_employees | Table_type |
+-----+-----+
| current_dept_emp    | VIEW      |
| dept_emp_latest_date | VIEW      |
+-----+-----+
```

2 rows in set (0,002 sec)

Adaptar y verificar en postgres:

Para crear las vistas en PostgreSQL se utiliza la instrucción **CREATE OR REPLACE VIEW**, que permite crear una nueva vista o reemplazar su definición si ya existe.

```
pdb_employees=# CREATE OR REPLACE VIEW dept_emp_latest_date AS
```

```
SELECT
```

```
    dept_emp.emp_no,
```

```
    MAX(dept_emp.from_date) AS from_date,
```

```
    MAX(dept_emp.to_date) AS to_date
```

```
FROM dept_emp
```

```
GROUP BY dept_emp.emp_no;
```

```
CREATE VIEW
```

```
pdb_employees=# CREATE OR REPLACE VIEW current_dept_emp AS
```

```
SELECT
```

```
    l.emp_no,
```

```
    d.dept_no,
```

```
    l.from_date,
```

```
    l.to_date
```

```
FROM dept_emp AS d
```

```
JOIN dept_emp_latest_date AS l
  ON d.emp_no = l.emp_no
  AND d.from_date = l.from_date
  AND l.to_date = d.to_date;
CREATE VIEW
```

Verificamos las vistas con \dv

```
pdb_employees=# \dv
```

Listado de vistas

Esquema	Nombre	Tipo	Dueño
public	current_dept_emp	vista	guillermo
public	dept_emp_latest_date	vista	guillermo

(2 filas)

Probamos que las vistas funcionen haciendo una consulta

```
pdb_employees=# SELECT * FROM dept_emp_latest_date LIMIT 10;
```

emp_no	from_date	to_date
10001	1986-06-26	9999-01-01
10002	1996-08-03	9999-01-01
10003	1995-12-03	9999-01-01
10004	1986-12-01	9999-01-01
10005	1989-09-12	9999-01-01
10006	1990-08-05	9999-01-01
10007	1989-02-10	9999-01-01
10008	1998-03-11	2000-07-31
10009	1985-02-18	9999-01-01

10010 | 2000-06-26 | 9999-01-01

(10 filas)

pdb_employees=# SELECT * FROM current_dept_emp LIMIT 10;

emp_no | dept_no | from_date | to_date

-----+-----+-----+-----

10004 | d004 | 1986-12-01 | 9999-01-01

10018 | d004 | 1992-07-29 | 9999-01-01

10020 | d004 | 1997-12-30 | 9999-01-01

10025 | d005 | 1987-08-17 | 1997-10-15

10027 | d005 | 1995-04-02 | 9999-01-01

10033 | d006 | 1987-03-18 | 1993-03-24

10037 | d005 | 1990-12-05 | 9999-01-01

10038 | d009 | 1989-09-20 | 9999-01-01

10043 | d005 | 1990-10-20 | 9999-01-01

10046 | d008 | 1992-06-20 | 9999-01-01

(10 filas)

Punto 3: Consultas de Verificación

Conteo

Consultamos el numero de registros en ambas bases de datos.

MariaDB:

MariaDB [employees]> SELECT 'departments' AS tabla, COUNT(*) AS filas FROM departments

-> UNION ALL

-> SELECT 'employees', COUNT(*) FROM employees

-> UNION ALL

```

-> SELECT 'dept_emp', COUNT(*) FROM dept_emp
-> UNION ALL
-> SELECT 'dept_manager', COUNT(*) FROM dept_manager
-> UNION ALL
-> SELECT 'salaries', COUNT(*) FROM salaries
-> UNION ALL
-> SELECT 'titles', COUNT(*) FROM titles;

```

```

+-----+-----+
| tabla   | filas |
+-----+-----+
| departments |    9 |
| employees   | 300024 |
| dept_emp    | 331603 |
| dept_manager |    24 |
| salaries    | 2844047 |
| titles      | 443308 |
+-----+-----+
6 rows in set (3,586 sec)

```

Postgres

```

pdb_employees=# SELECT 'departments' AS tabla, COUNT(*) AS filas FROM departments
UNION ALL
SELECT 'employees', COUNT(*) FROM employees
UNION ALL
SELECT 'dept_emp', COUNT(*) FROM dept_emp
UNION ALL

```

```

SELECT 'dept_manager', COUNT(*) FROM dept_manager
UNION ALL
SELECT 'salaries', COUNT(*) FROM salaries
UNION ALL
SELECT 'titles', COUNT(*) FROM titles;

```

```

      tabla      | filas
-----+-----
departments      |      9
employees         | 300024
dept_emp          | 331603
dept_manager      |      24
salaries          | 2844047
titles            | 443308
(6 filas)

```

Integridad

Para comprobar la integridad referencial de la relación entre empleados y departamentos se ejecuta la siguiente consulta en ambas bases de datos

La consulta utiliza un **LEFT JOIN** entre las tablas **dept_emp** y **employees**, relacionando ambas mediante el campo **emp_no**, que corresponde al identificador único del empleado. El objetivo es encontrar registros en **dept_emp** que no tengan un empleado asociado dentro de la tabla **employees**.

La condición **WHERE e.emp_no IS NULL** filtra únicamente aquellos registros donde no existe coincidencia en la tabla de empleados. Si el resultado es mayor a cero, indicaría la presencia de datos huérfanos, es decir, relaciones incompletas dentro de la base de datos.

MariaDB:

```
MariaDB [employees]> SELECT COUNT(*) AS empleados_huerfanos
```

```
-> FROM dept_emp de
```

```
-> LEFT JOIN employees e ON de.emp_no = e.emp_no
```

```
-> WHERE e.emp_no IS NULL;
```

```
+-----+
```

```
| empleados_huerfanos |
```

```
+-----+
```

```
|          0 |
```

```
+-----+
```

```
1 row in set (0,713 sec)
```

Postgres

```
pdb_employees=# SELECT COUNT(*) AS empleados_huerfanos
```

```
FROM dept_emp de
```

```
LEFT JOIN employees e ON de.emp_no = e.emp_no
```

```
WHERE e.emp_no IS NULL;
```

```
empleados_huerfanos
```

```
-----
```

```
0
```

```
(1 fila)
```

Los resultados en ambas son 0 , esto confirma que todos los registros de la tabla **dept_emp** tienen un empleado asociado y que la relación entre ambas tablas se conservó correctamente durante la migración.

Checksums

Como segunda validación se realiza una comparación de valores agregados utilizando la siguiente consulta en ambas db.

La función **COUNT(*)** permite obtener la cantidad total de registros almacenados en la tabla **employees**, mientras que **SUM(emp_no)** calcula la suma de todos los identificadores de empleados. La combinación de ambos valores funciona como una comprobación de consistencia, ya que permite comparar una cantidad de registros y un valor acumulado entre la base de datos origen y destino.

MariaDB:

MariaDB [employees]> SELECT

-> COUNT(*) AS filas,

-> SUM(emp_no) AS suma_emp_no

-> FROM employees;

```
+-----+-----+
| filas | suma_emp_no |
+-----+-----+
| 300024 | 76002608740 |
+-----+-----+

1 row in set (0,046 sec)
```

Postgres:

pdb_employees=# SELECT

COUNT(*) AS filas,

SUM(emp_no) AS suma_emp_no

FROM employees;

```
 filas | suma_emp_no
-----+-----
 300024 | 76002608740

(1 fila)
```

La coincidencia exacta de ambos valores confirma que la tabla **employees** fue migrada correctamente, manteniendo la misma cantidad de registros y los mismos identificadores que la base de datos original. Esto proporciona evidencia adicional de que la transferencia de datos entre MariaDB y PostgreSQL se realizó sin alteraciones.